

CSA0904 - JAVA PROGRAMMING

Scenario 1: Library Book Management System

Code:

```
import java.util.*;
class Book {
    String isbn, title, author;
    int year;
    Book(String isbn, String title, String author, int year) {
        this.isbn = isbn;
        this.title = title;
        this.author = author;
        this.year = year;
    }
    public boolean equals(Object obj) {
        if (this == obj) return true;
        if (!(obj instanceof Book)) return false;
        Book b = (Book) obj;
        return isbn.equals(b.isbn);
    }
    public int hashCode() {
        return isbn.hashCode();
    }
    public String toString() {
        return isbn + " | " + title + " | " + author + " | " + year;
    }
}
class YearComparator implements Comparator<Book> {
    public int compare(Book a, Book b) {
        return b.year - a.year;
    }
}
public class LibraryManagement {
    public static void main(String[] args) {
        HashSet<Book> books = new HashSet<>();
        books.add(new Book("101", "Java", "James", 2020));
        books.add(new Book("102", "Python", "Guido", 2022));
        books.add(new Book("103", "C++", "Bjarne", 2018));
        books.add(new Book("101", "Duplicate", "ABC", 2023));

        String removeISBN = "103";
```

```
Iterator<Book> it = books.iterator();
while (it.hasNext()) {
    Book b = it.next();
    if (b.isbn.equals(removeISBN))
        it.remove();
}
```

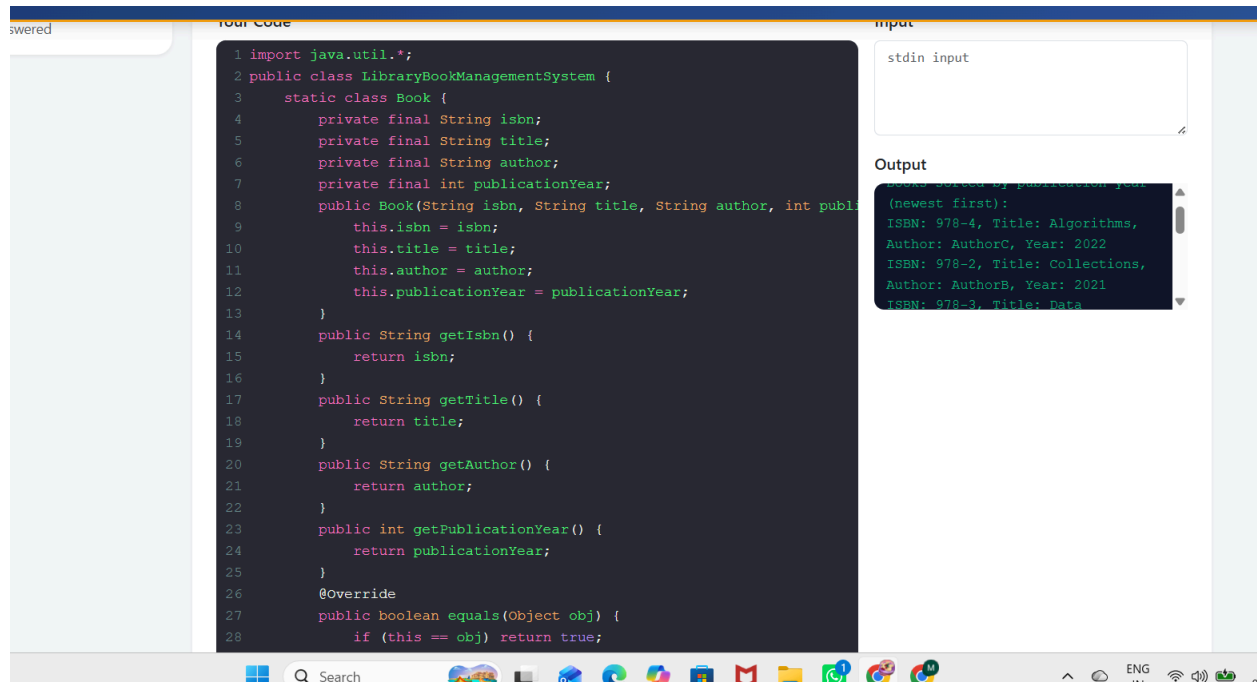
```
ArrayList<Book> list = new ArrayList<>(books);
Collections.sort(list, new YearComparator());
```

```
System.out.println("Books Sorted by Year:");
```

```
Iterator<Book> itr = list.iterator();
while (itr.hasNext())
    System.out.println(itr.next());
```

```
System.out.println("\nBooks by Guido");
```

```
itr = list.iterator();
while (itr.hasNext()) {
    Book b = itr.next();
    if (b.author.equalsIgnoreCase("Guido"))
        System.out.println(b);
}
}
```



Scenario 2: Student Grade Analyzer

Code:

```
import java.util.*;

class Student {
    int id;
    String name;
    HashMap<String, Integer> grades = new HashMap<>();

    Student(int id, String name) {
        this.id = id;
        this.name = name;
    }

    double average() {
        int sum = 0;
        for (int g : grades.values())
            sum += g;
        return (double) sum / grades.size();
    }

    int failedSubjects() {
```

```

        int count = 0;
        for (int g : grades.values())
            if (g < 40)
                count++;
        return count;
    }

    public String toString() {
        return id + " " + name + " Average=" + average();
    }
}

class AvgComparator implements Comparator<Student> {
    public int compare(Student a, Student b) {
        return Double.compare(b.average(), a.average());
    }
}

public class StudentAnalyzer {
    public static void main(String[] args) {

        ArrayList<Student> students = new ArrayList<>();

        Student s1 = new Student(1, "Ram");
        s1.grades.put("Math", 90);
        s1.grades.put("Java", 80);
        s1.grades.put("Python", 70);

        Student s2 = new Student(2, "Sita");
        s2.grades.put("Math", 20);
        s2.grades.put("Java", 30);
        s2.grades.put("Python", 35);

        Student s3 = new Student(3, "John");
        s3.grades.put("Math", 95);
        s3.grades.put("Java", 90);
        s3.grades.put("Python", 98);

        Student s4 = new Student(4, "David");
        s4.grades.put("Math", 75);
        s4.grades.put("Java", 65);
        s4.grades.put("Python", 85);

        students.add(s1);
    }
}

```

```
students.add(s2);
students.add(s3);
students.add(s4);
```

```
Iterator<Student> it = students.iterator();
```

```
while (it.hasNext()) {
    Student s = it.next();
    if (s.failedSubjects() > 2)
        it.remove();
}
```

```
Collections.sort(students, new AvgComparator());
```

```
System.out.println("Top Students");
```

```
for (int i = 0; i < Math.min(3, students.size()); i++)
    System.out.println(students.get(i));
```

```
}
}
```

```
1 class Student {
2     String id;
3     String name;
4     HashMap<String, Integer> grades;
5     Student(String id, String name) {
6         this.id = id;
7         this.name = name;
8         this.grades = new HashMap<>();
9     }
10    void addGrade(String subject, int grade) {
11        grades.put(subject, grade);
12    }
13    double averageGrade() {
14        if (grades.isEmpty()) return 0.0;
15        int sum = 0;
16        for (int g : grades.values()) sum += g;
17        return (double) sum / grades.size();
18    }
19 }
20 ArrayList<Student> students = new ArrayList<>();
21 Student s1 = new Student("S01", "Asha");
22 s1.addGrade("Math", 55);
23 s1.addGrade("Science", 60);
24 s1.addGrade("English", 38);
25 students.add(s1);
26 Student s2 = new Student("S02", "Rahul");
27 s2.addGrade("Math", 20);
28 s2.addGrade("Science", 35);
```

stdin input

Output

System error.

Scenario 3: Navigation History

Code :

```
import java.util.*;
```

```
public class BrowserNavigation {

    Stack<String> back = new Stack<>();
    Stack<String> forward = new Stack<>();

    void visitPage(String url) {

        if (!back.isEmpty())
            System.out.println("Leaving: " + back.peek());

        back.push(url);
        forward.clear();
    }

    void goBack() {

        if (back.size() <= 1) {
            System.out.println("Cannot Go Back");
            return;
        }

        forward.push(back.pop());

        System.out.println("Current Page: " + back.peek());
    }

    void goForward() {

        if (forward.isEmpty()) {
            System.out.println("Cannot Go Forward");
            return;
        }

        back.push(forward.pop());

        System.out.println("Current Page: " + back.peek());
    }

    void display() {
```

```
        System.out.println("\nBack History");

        Iterator<String> it = back.iterator();
        while (it.hasNext())
            System.out.println(it.next());

        System.out.println("\nForward History");

        it = forward.iterator();
        while (it.hasNext())
            System.out.println(it.next());

        System.out.println("\nCurrent Page: " + back.peek());
    }

    public static void main(String[] args) {

        BrowserNavigation b = new BrowserNavigation();

        b.visitPage("google.com");
        b.visitPage("youtube.com");
        b.visitPage("github.com");

        b.goBack();
        b.goBack();
        b.goForward();

        b.visitPage("chatgpt.com");

        b.display();
    }
}
```

roll System

✓

- Use Stack<String> for back history
- Use Stack<String> for forward history
- Use Iterator to display all pages in both stacks without modifying them
- Handle edge cases (empty stack, end of navigation)

Your Code

```

1 import java.util.ArrayDeque;
2 import java.util.Deque;
3 public class BrowserNavigation {
4     private final Deque<String> backHistory;
5     private final Deque<String> forwardHistory;
6     private String currentPage;
7     public BrowserNavigation(String homepage) {
8         this.backHistory = new ArrayDeque<>();
9         this.forwardHistory = new ArrayDeque<>();
10        this.currentPage = homepage;
11    }
12    public void visit(String url) {
13        if (currentPage != null) {
14            backHistory.push(currentPage);
15        }
16        currentPage = url;
17        forwardHistory.clear();
18        System.out.println("Visited: " + currentPage);
19    }
20    public void back() {
21        if (backHistory.isEmpty()) {

```

Input

stdin input

Output

```

=====
Current Page: https://google.com
Back History (Top to Bottom): []
Forward History (Top to Bottom):
[]
=====

```

Scenario 4: Employee Payroll System

Code:

```
import java.util.ArrayDeque;
```

```
import java.util.Deque;
```

```
public class BrowserNavigation {
```

```
    private final Deque<String> backHistory;
```

```
    private final Deque<String> forwardHistory;
```

```
    private String currentPage;
```

```
    public BrowserNavigation(String homepage) {
```

```
        this.backHistory = new ArrayDeque<>();
```

```
        this.forwardHistory = new ArrayDeque<>();
```

```
        this.currentPage = homepage;
```

```
    }
```

```
    public void visit(String url) {
```

```
        if (currentPage != null) {
```

```
            backHistory.push(currentPage); // Push old page to history stack
```

```
        }
```

```
        currentPage = url;
```

```
        forwardHistory.clear(); // Visiting a new page resets forward navigation
```

```
        System.out.println("Visited: " + currentPage);
```

```
    }
```

```
    public void back() {
```

```
        if (backHistory.isEmpty()) {
```

```

        System.out.println("⚠ Cannot go back. You are at the beginning of history.");
        return;
    }
    forwardHistory.push(currentPage); // Save current page to forward stack
    currentPage = backHistory.pop(); // Retrieve previous page
    System.out.println("Moved Back To: " + currentPage);
}

public void forward() {
    if (forwardHistory.isEmpty()) {
        System.out.println("⚠ Cannot go forward. No history ahead.");
        return;
    }
    backHistory.push(currentPage); // Save current page to back stack
    currentPage = forwardHistory.pop(); // Retrieve next page
    System.out.println("Moved Forward To: " + currentPage);
}

public void displayStatus() {
    System.out.println("\n===== BROWSER STATUS
=====");
    System.out.println("Current Page: " + currentPage);
    System.out.println("Back History (Top to Bottom): " + backHistory);
    System.out.println("Forward History (Top to Bottom): " + forwardHistory);

    System.out.println("=====\n");
}

public static void main(String[] args) {
    // Start browser session at Google
    BrowserNavigation browser = new BrowserNavigation("https://google.com");
    browser.displayStatus();

    // 1. Simulate visiting new websites
    browser.visit("https://wikipedia.org");
    browser.visit("https://www.github.com");
    browser.displayStatus();

    // 2. Go back one page
    browser.back();
}

```

```
browser.displayStatus();
```

```
// 3. Go back another page
```

```
browser.back();
```

```
browser.displayStatus();
```

```
// 4. Try going forward again
```

```
browser.forward();
```

```
browser.displayStatus();
```

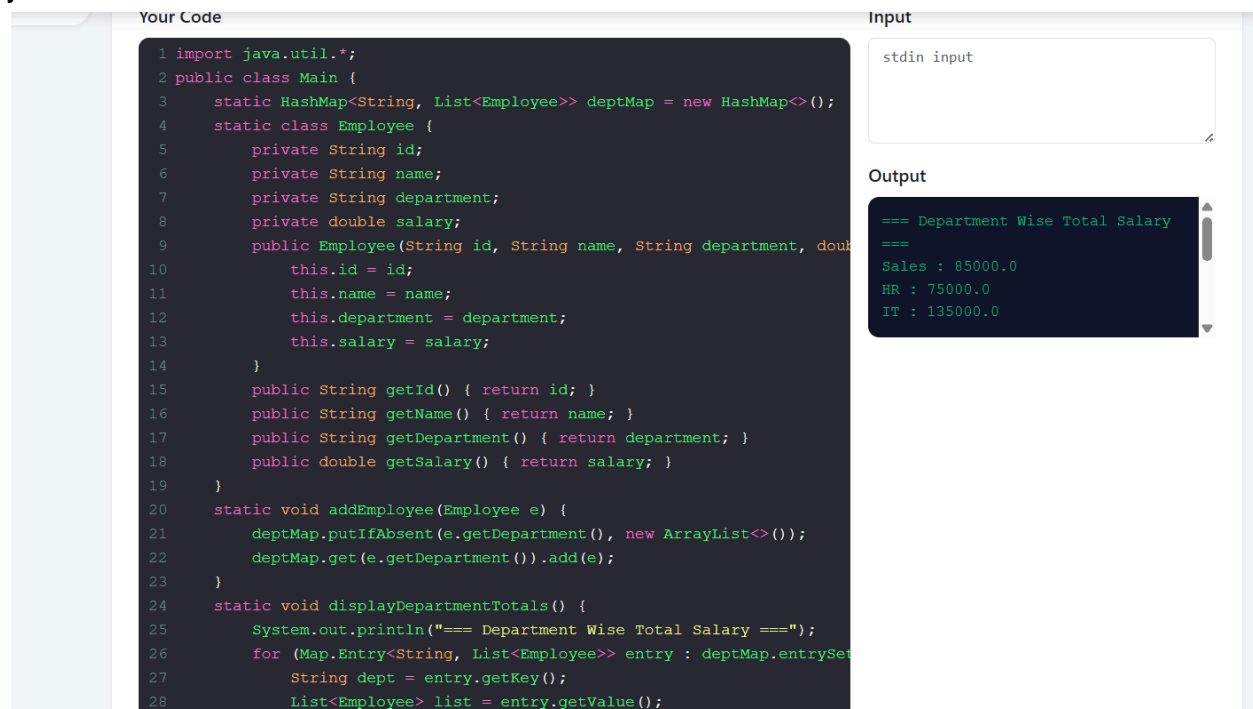
```
// 5. Break forward history path by visiting a clean URL
```

```
browser.visit("https://www.stackoverflow.com");
```

```
browser.displayStatus();
```

```
}
```

```
}
```



The screenshot shows a Java IDE with three panels: 'Your Code', 'Input', and 'Output'. The 'Your Code' panel contains Java code for a program that calculates department-wise total salaries. The 'Input' panel shows 'stdin input'. The 'Output' panel shows the program's output, which is the department-wise total salary for Sales, HR, and IT departments.

```
1 import java.util.*;
2 public class Main {
3     static HashMap<String, List<Employee>> deptMap = new HashMap<>();
4     static class Employee {
5         private String id;
6         private String name;
7         private String department;
8         private double salary;
9         public Employee(String id, String name, String department, double salary) {
10             this.id = id;
11             this.name = name;
12             this.department = department;
13             this.salary = salary;
14         }
15         public String getId() { return id; }
16         public String getName() { return name; }
17         public String getDepartment() { return department; }
18         public double getSalary() { return salary; }
19     }
20     static void addEmployee(Employee e) {
21         deptMap.putIfAbsent(e.getDepartment(), new ArrayList<>());
22         deptMap.get(e.getDepartment()).add(e);
23     }
24     static void displayDepartmentTotals() {
25         System.out.println("=== Department Wise Total Salary ===");
26         for (Map.Entry<String, List<Employee>> entry : deptMap.entrySet()) {
27             String dept = entry.getKey();
28             List<Employee> list = entry.getValue();
```

Input

```
stdin input
```

Output

```
=== Department Wise Total Salary ===
Sales : 85000.0
HR : 75000.0
IT : 135000.0
```